

Investigating Fault-Induced Misconfiguration of RISC-V PMP and Its Consequences for isolated software execution environments

Kévin Quénéhervé¹, Philippe Tanguy¹, Rachid Dafali², and Vianney Lapôtre¹

¹ Université Bretagne Sud, UMR 6285, Lab-STICC, Lorient, France
`firstname.surname@univ-ubs.fr`

² DGA MI, Rennes, France `rachid.dafali@def.gouv.fr`

Abstract. We study Fault Injection Attacks (FIA) targeting PMP. We analyse an extensive clock-glitching campaign on several RISC-V cores. We provide a fine-grained, bit-level characterization of how injected faults corrupt the PMP configuration registers. Then, we develop a theoretical impact analysis of how such physical attack would translate into isolation violations across representative PMP-based software stacks such as TEEs, MultiZone, and Real-Time Operating Systems (RTOS). Our results show that the fragility of PMP under fault injection does not remain confined to the hardware level but propagates to every isolation layer that depends on it. For instance, across three RISC-V cores (Ibex, VexRISCV and CVA6), more than half of the observed fault effects are bit-flips in the PMP registers that extend a protected region and create memory overlaps. Finally, we highlight 11 exploitation scenarios.

Keywords: Hardware Security · Fault Injection Attacks · RISC-V · Physical Memory Protection · FPGA.

1 Introduction

For decades, embedded systems have relied on hardware-enforced isolation mechanisms to protect sensitive assets and ensure system integrity. With the increasing deployment of security-critical applications, memory isolation and hardware access-control mechanisms have become fundamental building blocks of modern System on Chip (SoC) architectures [20]. In this context, the open and modular RISC-V architecture has gained significant attention, enabling flexible integration of custom security features.

The guarantees provided by these mechanisms can, however, be undermined by physical attacks. Fault Injection Attacks (FIA) [11,10] are among the most serious threats to hardware security. By introducing transient disturbances such as clock glitches [9], electromagnetic (EM) pulses [25], or voltage variations, an attacker can induce erroneous computations and bypass critical security checks [26,18]. Such techniques have been used to compromise cryptographic implementations, bypass authentication, corrupt the program counter [24], and defeat secure boot [26].

Within the RISC-V ecosystem, the Physical Memory Protection (PMP) mechanism is the main hardware feature enforcing spatial memory isolation between privilege levels and software components. A broad range of *isolated software execution environments* schemes build directly on PMP, yet exploit it differently: some reconfigure PMP entries at every context or enclave switch, some pre-allocate fixed protected zones, and others lock entries once at boot [15,13,3,4,7,22]. PMP is thus a shared substrate from which many strategies, such as TEEs, firmware monitors, and real-time operating systems (RTOSes) inherit their security guarantees. Despite this central role, the robustness of PMP against fault injection has received far less attention than its functional correctness, leaving the resilience of every software layer built upon it largely unexplored.

Physical attacks are increasingly critical as open-source RISC-V cores reach security-sensitive applications. Recent vulnerabilities already expose flaws in MMU/PMP enforcement: wrong exception type on a PMP violation during translation (CVE-2022-34641, CVA6 and BOOM) and spurious faults on valid SV39 translations (CVE-2025-50897, BOOMv1.2), hinting that this interplay may open new attack surfaces under fault conditions. The few studies on PMP under faults reinforce this concern: Nashimoto et al. [18] showed that PMP configuration registers can be modified on a RISC-V processor through clock-glitching FIA, and end-to-end compromises have since been demonstrated on the Keystone TEE [19]. These works, however, each follow a single attack path and characterize only the fault effects it requires. Yet a systematic, fine-grained understanding of *where* and *how* PMP enforcement fails which configuration bits flip, how exceptions are handled, and whether access controls can be transiently bypassed is still missing, leaving the guarantees inherited by the RTOS, firmware, and TEE layers unassessed.

To address this gap, this paper investigates the effect of clock-glitching fault injection on PMP at the bit level, across several RISC-V cores implemented as FPGA softcores, and analyzes how the resulting corruption propagates to the software isolation layers that depend on PMP. The main contributions of this paper are as follows:

- (i) We present a comprehensive experimental study of the robustness of the RISC-V PMP mechanism against clock-glitching fault injection. We characterize fault effects across three various RISC-V cores and provide a fine-grained, bit-level analysis of how injected faults corrupt the PMP configuration registers. We further propose a taxonomy of the observed faults and analyze their spatial and temporal distribution.
- (ii) We assess the security implications of these faults on the software layers that build memory isolation upon PMP. We show how bit-level corruption of PMP entries translates into concrete violations of memory isolation guarantees. Finally, we show the security impact on software such as RTOSes and TEE-like environments. We characterize the resulting exploitation surface rather than instantiating a specific end-to-end attack, which is left to future work.

This paper is organized as follows: Section 2 provides the necessary background. Section 3 introduces the threat model, the attack scenarios, and the ex-

perimental setup. Section 4 presents the results and discusses their exploitability and security implications. Finally, Section 5 concludes the paper.

2 Background

This section provides general information on RISC-V cores, the physical memory protection (PMP) mechanism and their use in different categories of software.

2.1 Evaluated RISC-V Cores

To evaluate the impact of fault injection across different architectural designs, we consider three widely used open-source RISC-V cores: CVA6 [27], Ibex[2], and VexRISCV[5]. Their main characteristics are summarized in Table 1.

Table 1: Comparison of open-source RISC-V cores: CVA6, Ibex, and VexRISCV

Feature	CVA6 (Ariane)	Ibex	VexRISCV
ISA	RV64IMAC	RV32IMC	RV32IM (configurable)
Pipeline	6-stage	2-3 stage	2-5 stage (configurable)
Execution	In-order, single-issue	In-order	In-order
Target	Application-class	Embedded / security	FPGA softcore
MMU	Yes	No	Optional
OS Support	Linux capable	Bare-metal / RTOS	RTOS / limited Linux
Caches	L1 I/D cache	Optional	Configurable
Branch Prediction	Yes (BTB + BHT)	Simple	Configurable
Complexity	High	Low	Medium (modular)
Use Case	SoC Linux, research	Secure embedded	Custom CPU / FPGA

As shown in Table 1, these cores exhibit significant differences in terms of pipeline depth, complexity, and supported features. CVA6 [27] is a 64-bit application-class processor (RV64IMAC) with a six-stage in-order, single-issue pipeline, L1 instruction and data caches, branch prediction, and full MMU support. Its Linux capability makes it representative of complex SoC designs. In contrast, Ibex [2] is a lightweight 32-bit core (RV32IMC) targeting embedded and security-focused applications: it features a shallow two- to three-stage pipeline, no MMU, and optional caches, and typically runs bare-metal or RTOS workloads. Finally, VexRISCV [5] is a highly configurable 32-bit core (RV32IM) tailored to FPGA softcore designs, with a pipeline depth, cache configuration, and optional MMU that can all be tuned to trade performance against resource usage; it supports RTOS and, in suitable configurations, limited Linux deployments.

Together, these cores span a representative spectrum from a complex, Linux-capable application processor (CVA6) to a minimal secure embedded core (Ibex) and a modular FPGA-oriented design (VexRISCV) allowing us to observe how fault effects on PMP vary with microarchitectural complexity.

2.2 Physical Memory Protection Mechanism in RISC-V

The RISC-V processors can support up to 16 Physical Memory Protection (PMP) regions. These regions are configured through Control and Status Registers (CSRs), which must be accessed in machine mode (M-mode), the highest privilege level. PMP uses two types of registers: configuration registers called `pmpcfg` and address registers called `pmpaddr`. The `pmpcfg` registers define access privileges for memory regions, while the `pmpaddr` registers define the corresponding memory addresses. The number and grouping of these registers depend on whether the core is 32-bit or 64-bit.

According to the RISC-V specification [6], each PMP region is configured through an 8-bit field denoted as `pmpicfg`, where $0 \leq i \leq 15$. On 64-bit architectures, eight such configuration fields are aggregated into a single 64-bit control and status register (CSR), referred to as `pmpcfgi`, whereas on 32-bit architectures, four configuration fields are grouped into a 32-bit register. For instance, the 64-bit CVA6 core provides two configuration registers, namely `pmpcfg0` and `pmpcfg2`. In contrast, a conventional 32-bit RISC-V core typically implements four configuration registers, ranging from `pmpcfg0` to `pmpcfg3`. Each `pmpicfg` field encodes a set of control bits, namely L, A, X, W, and R. The R, W, and X bits respectively enable read, write, and execute privileges. The L (Lock) bit, when asserted, prevents any subsequent modification of the associated PMP entry until a hardware reset occurs; furthermore, when set, the corresponding access privileges are enforced across all privilege levels.

The A (Addressing-mode) field specifies the addressing mode used to define the boundaries of the protected memory region. This field supports four distinct modes: OFF (A=00) disables the PMP entry, leaving the region unprotected; TOR (Top Of Range, A=01) uses two consecutive registers, where `pmpaddri` forms the upper bound and `pmpaddri-1` forms the lower bound of the region; NA4 (Naturally Aligned Four-bytes, A=10) enforces a fixed, naturally aligned 4-byte region; and NAPOT (Naturally Aligned Power-Of-Two, A=11) allows a single `pmpaddri` register to encode both the base address and the size of the region.

In NAPOT mode, the encoding of `pmpaddr` depends on the target architecture. On 32-bit RISC-V core, the register is 32 bits wide and stores the word-aligned address shifted right by two bits, yielding 30 effective address bits [31:2]. On 64-bit RISC-V core, the register is 64 bits wide; bits [63:54] are reserved (WPRI³), and bits [53:0] encode the physical address shifted right by two, supporting physical addresses up to 56 bits.

In both cases, the register encodes both the base address and the region size within a single value. The boundary between the two fields is determined by y , the number of consecutive trailing ones in `pmpaddr`. Specifically, the y least-significant bits are set to 1, the next bit (bit y) is 0, and the remaining upper bits encode the base address of the naturally aligned region. The protected region size is then 2^{y+3} bytes, with $y = 0$ corresponding to the minimum encodable size of

³ WPRI: Writes Preserved, Reads Ignored—a RISC-V convention for reserved fields that must be preserved on write and may return any value on read.

Table 2: Taxonomy of isolated software execution environments based on PMP.

Class	Description	Software	Mode	Protected domains	PMP config.
C1	Dynamic TEEs	Keystone [15], Penglai [13]	M-mode (Security Monitor)	One per active enclave	On every enclave entry/exit
C2	Zone partitioning	MultiZone [3]	M-mode (nanokernel)	Fixed, pre-allocated zones	Full table reload per zone switch
C3	Static firmware monitors	OpenSBI [4]	M-mode (firmware)	Two (firmware vs. rest)	Set once at boot, then locked
C4	RTOS process isolation	TockOS [22], Zephyr [7]	M/S-mode (kernel)	One per task/process	On every context switch

8 bytes. For example, a 128-bytes region ($y = 4$) is encoded as `aaaa...a01111`, where the four trailing ones define the region size and the `a`-bits encode the base address.

2.3 Isolated software execution environments based on PMP

Across all RISC-V systems, PMP is the common hardware primitive used to enforce spatial memory isolation between privilege levels. However, the software stacks built on top of PMP exploit it in markedly different ways. Three design dimensions distinguish these stacks: (i) the number of protected domains that coexist at runtime, (ii) the frequency at which PMP entries are reconfigured (static and locked at boot versus rewritten on every context switch), and (iii) the PMP addressing mode employed. Building on these dimensions, we propose a taxonomy that classifies isolated software execution environments based on PMP into four classes. Table 2 summarizes their respective trade-offs.

Class 1: Dynamic Trusted Execution Environments (TEEs). RISC-V TEEs use PMP as the basic primitive to isolate mutually distrustful enclaves from an untrusted rich operating system. Frameworks such as Keystone [15] and Penglai [13] manage PMP entries dynamically from Machine mode (M-mode) through a Security Monitor. Keystone keeps a couple of static entries (one for the monitor, one for the untrusted/OS memory) and allocates one entry per active enclave, reconfiguring the PMP registers on every enclave entry and exit to keep execution domains strictly separated. Because the hardware offers only a limited number of PMP entries (typically 8 to 16), this approach hits a scalability ceiling: Penglai removes it with a Guarded Page Table that decouples the number of concurrent enclaves from the PMP entry count.

Class 2: Hardware-Enforced Zone Partitioning. Zone-partitioning models such as Hex Five’s MultiZone [3] take a static, pre-allocated approach instead

of a dynamic enclave lifecycle. MultiZone splits the physical address space into a fixed number of isolated, immutable *zones*, each mapped to a fixed subset of PMP entries at build or initialization time. A minimal M-mode microkernel reloads the whole PMP table at every zone-boundary transition, rather than updating single entries. This fully isolates legacy applications or untrusted third-party code (e.g., a network stack) without an operating system or page tables, at the cost of a fixed number of domains.

Class 3: Static Firmware and Supervisor Monitors. Static firmware layers aim for a near-zero runtime cost: they set fixed partitions once at boot and then leave them unchanged. OpenSBI [4], the reference implementation of the RISC-V Supervisor Binary Interface, is the canonical example. Running in M-mode as the first boot stage, it sets its memory boundaries early and, in the typical static case, locks them through the PMP lock bit (L). Usually only two entries are needed: one giving the firmware exclusive rights over its own code and data, and one exposing the rest of memory to lower privilege modes (e.g., Supervisor mode for Linux). Once locked, these regions cannot be changed by lower modes and add no runtime overhead. OpenSBI generalizes this scheme through its *domain* abstraction, which splits harts and memory regions into system-level isolation domains enforced via PMP.

Class 4: Real-Time and Embedded Operating Systems (RTOS). On resource-constrained microcontrollers, modern RTOSes use PMP instead of heavy page tables to enforce process-level memory safety. Systems such as TockOS [22] and the secure profiles of Zephyr [7] rewrite the PMP configuration at every task context switch to apply the principle of least privilege. TockOS uses TOR addressing to confine each user process to its own stack and code regions, while Zephyr relies on PMP for both userspace isolation and a hardware stack guard. Moving these boundaries between tasks prevents a faulty driver or a compromised application thread from corrupting kernel structures or interfering with other high-criticality tasks.

3 Experiments

This section describes the experimental setup and the scenario for fault evaluation during PMP configuration.

3.1 Threat Model

We assume that the attacker has physical access to the target device and can perform FIA by relying on clock manipulation. We consider that the attacker program does not have the privilege to modify the PMP configuration. We consider the attacker is able to precisely synchronize its attacks by relying, for instance, on GPIO activity. In this study, we do neither consider side-channel analysis nor reverse engineering, software, and network-based attacks.

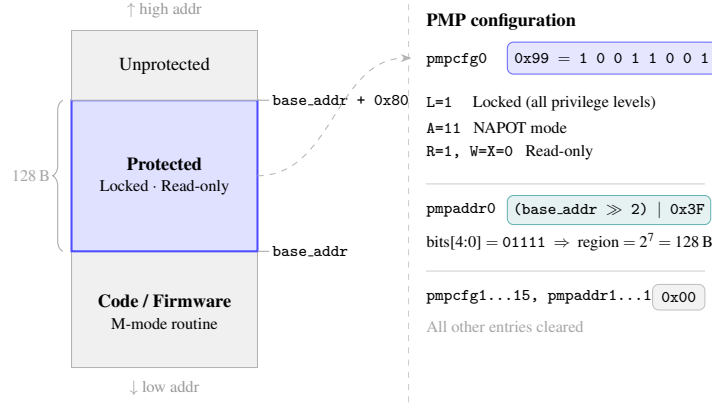


Fig. 1: PMP memory layout: 128-byte read-only locked region in NAPOT mode.

3.2 Evaluation scenario

We implement an M-mode routine to configure a protected PMP region in NAPOT mode, following [18,21], using the SiFive Freedom Metal PMP library [1]. Figure 1 illustrates the resulting PMP configuration for a locked, read-only 128-byte memory region starting at `0x20000600`. As shown in the figure, two CSR registers are programmed:

- `pmpcfg0` is set to `0x99`, encoding `L=1` (locked), `A=11` (NAPOT mode), and `R=1`, `W=X=0` (read-only privileges).
- `pmpaddr0` is set to the NAPOT-encoded base address, where bits `[4:0]` are set to `0b01111`, defining a 128-bytes region (2^7).
- All remaining PMP entries (`pmpcfg2`, `pmpaddr1`–`pmpaddr15`) are initialized to `0x76`.

The `L` bit enforces the configuration across all privilege levels and prevents any further update until the next system reset, a key security property of PMP regions. Prior to configuration, all PMP registers are initialized with a fix value. This choice is deliberate: it allows distinguishing between two fault classes during fault injection experiments *no-update* faults, where the register retains its fix initial value, and *reset* faults, where the register is cleared to zero. The firmware is compiled with `-Os` to satisfy embedded memory constraints. A trigger signal is emitted via a GPIO immediately before the PMP configuration sequence begins, providing a precise temporal reference for fault injection.

The configuration sequence proceeds in three steps: 1) compute the NAPOT-encoded address from the base address (detailed in Figure 3 in Appendix A.1), 2) write it to `pmpaddr0` via `csrw`, then 3) set `pmpcfg0` via `csrs`. Each of these intermediate steps constitutes a potential fault injection target. After configuration, the software attempts a write access to the protected region. Under correct

execution, this access triggers a memory access fault exception, confirming the protection is active. A fault-induced corruption of the configuration sequence may instead silently allow the write to succeed, resulting in unauthorized memory access. The final PMP register state is reported via the serial output after each execution, enabling post-experiment analysis.

3.3 Evaluation setup

The device under test constitutes an SoC generated with the LiteX framework⁴ release 2022.08. The three designs are synthesized with Vivado 2019.1 and deployed on the same Digilent Arty A7-100T FPGA board [8]. Each SoC integrates one of the three RISC-V cores described in section 2, along with on-chip RAM, a ROM containing the firmware described in section 3.2, general-purpose I/O (GPIO), and a UART interface for communication. The three SoCs are therefore identical except for their RISC-V core. Three experimental configurations are evaluated on each core.

Fault injections are performed using the ChipWhisperer Lite⁵ platform. A host computer orchestrates the experiments, including bitstream loading, the execution of characterization. In this study, clock glitching is obtained by **XORing** the original *clock signal* with a *clock XOR*, like presented in [21].

The glitch is configured through four parameters: 1) *Width*: percentage of a clock period $[-45, 45]$ controlling the glitch width. Wider glitches increase fault probability but also crash risk; 2) *Offset*: percentage of a clock period $[-45, 45]$ controlling the glitch placement within the glitched cycle; 3) *External Offset*: delay in clock cycles $[0, 2^{32}]$ between the trigger and the glitch, used to target specific instructions; 4) *Repeat*: number of consecutive glitched cycles $[1, 8192]$.

In this work, *Repeat* is fixed to 1 to isolate the effects of a single glitch, while *Width* and *Offset* are swept exhaustively over their full range with a step of 1, i.e. 91 values each. Analysis of the HDL waveform shows that the write operations to the registers enabling the PMP configuration take approximately 100 clock cycles; we therefore limit the scanning of the *External Offset* to this 100-cycle window.

An ideal trigger and full parameter control are used deliberately: removing synchronization uncertainty makes the reported rates ($91 \times 91 \times 101 = 836,381$ injections per configuration) characterize the fault effect itself rather than the triggering accuracy. This does not limit generality, since SCA-based synchronization is highly effective in realistic settings [19].

Each injection outcome is characterized by reading the PMP registers via a UART interface. The results are stored in a `.csv` dataset, where each entry corresponds to a single injection attempt.

Each record includes the injection parameters (*Width*, *Offset*, *External Offset*) and the resulting PMP register values (`pmpcfg0` and `pmpaddr0`). To systematically analyze the collected data, we classify the observed PMP register

⁴ <https://github.com/enjoy-digital/litex/tree/2022.08>

⁵ <https://rtfm.newae.com/Capture/ChipWhisperer-Lite/>

corruptions into seven distinct fault effect categories. The first three are bit-flip effects, which correspond to the corruption of one or more bits within the register: a single bit-flip affects exactly one bit, a double bit-flip affects two bits, and a multiple bit-flip (> 2) corrupts more than two bits simultaneously. The next two effects model forced value faults: a reset effect forces the entire register to zero ($0x00 \dots 0$), while a set effect forces it to all ones ($0xFF \dots F$), regardless of the intended written value. Finally, we consider two structural fault effects. A shift effect occurs when the correct data pattern is shifted within the register, resulting in an incorrect value being stored. A no-update effect models a stuck-at-previous-value behavior, where the register retains its last stored value and is not updated despite a write operation being issued.

4 Experimental Characterization of PMP Register Faults

4.1 Fault effects combinations

Table 3 summarizes the global outcomes of the fault injection campaigns across the three evaluated RISC-V cores. The Ibex core exhibits the highest number of

Table 3: Comparison of clock fault injection campaigns across three experimental RISC-V Cores – with 836,381 faults injected per campaign.

Core	Normal	Crash	PMP corruptions			
			G1	G2	G3	Total
Ibex	832,798	1,381	86	1,881	235	2,202
VexRiscv	787,828	48,510	0	40	3	43
CVA6	783,090	51,664	3	1,389	235	1,627

PMP corruptions (2,202), followed by CVA6 (1,627), whereas VexRISCV shows only 43 PMP corruptions. In contrast, VexRISCV produces a large number of system crashes (48,510), comparable to CVA6 (51,664), suggesting that faults tend to propagate toward global failures rather than a localized PMP corruption.

We classify the various fault injections in three categories per experiments as shown Table 3. The first group (G1) gathers faults that lead to *complex* effects leading, for instance, to the storage of *random* values into one or several PMP configuration registers, the storage of a faulty value into several PMP configuration registers or the storage of the correct value in a PMP configuration register which is not the targeted one. However, due to the unpredictability of G1 fault effects for an attacker, this group is not further analyzed in this work.

The second group (G2) gathers faults that impact a single register among all PMP configurations. In this group, either `pmpcfg0` or `pmpaddr0` are impacted (these two registers are the ones configured by the firmware) while others PMP

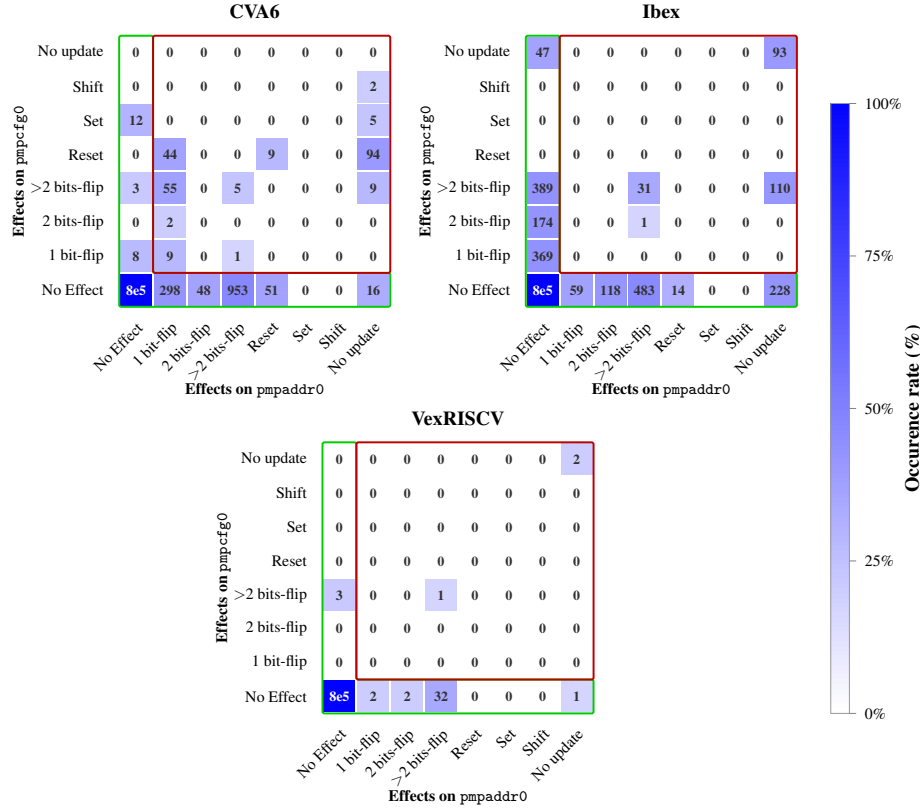


Fig. 2: Fault injection effect combinations on **RISC-V Core** at $25MHz$ - groups **G2** & **G3**.

configuration are not impacted. Each cell of Figure 2 reports the number of injection points producing a given pair of fault effects on the two registers; the first column and the last row give, for this group, the number of fault injections leading to a single-register effect. The color of each cell encodes the *occurrence rate*, defined as the proportion of these injections with respect to the total number of injections. We observe that for `pmpcfg0`, bit-set is the most common effect and for `pmpaddr0`, multiple bit-flips is on CVA6 core. The second and third most common effects are multiple bits-flip and register set, unlike the behavior observed on the Ibex and VexRISCV cores. Furthermore, we observe more no-update effect on both `pmpcfg0` and `pmpaddr0` on Ibex core.

From a security perspective, faults in G2 have direct implications for the applications described in section 2.3. In Class 3 including OpenSBI, where PMP entries are locked at boot and never modified, a fault corrupting `pmpcfg0`, for instance via a *set* effect granting full RWX privileges. It would bypass the firmware's static isolation without triggering any runtime reconfiguration, making the at-

tack particularly stealthy. In Class 2 including Multizone, where up to 16 zones each rely on dedicated PMP entries, a fault on `pmpaddr0` corrupting the base address or size of a zone could silently expand or shift that zone’s memory region, causing it to overlap with an adjacent zone and breaking inter-zone isolation. In addition, bit flips on `pmpcfg0` could alter access privileges or even unlock the protected area.

The third group (G3) gathers faults that impact both `pmpcfg0` and `pmpaddr0` but do not impact other PMP configuration registers. Figure 2 shows that a fault injection can lead to couple of effects in the two configured registers. These couple of effects are built around bit(s)-flip, register reset, set or value shift or no-update in the configured register on the CVA6 core. Unlike CVA6, Ibex and VexRISCV are not affected by register sets, resets and value shifts, but only by bit-flips and a few no-update faults.

The simultaneous corruption of both registers makes G3 faults particularly severe from a security standpoint. A fault jointly altering `pmpcfg0` and `pmpaddr0` can both change the access privileges of a PMP entry and relocate or resize the memory region it protects, compounding the impact of each individual corruption. In Class 3 including Keystone, such a combined fault occurring during an enclave entry or exit could simultaneously widen an enclave’s accessible memory range and elevate its privileges, potentially granting it access to the Security Monitor’s own memory region. In Class 4 including RTOSes, where PMP entries are rewritten at every context switch, a G3 fault could misattribute both the address range and the privilege set of a process.

In most configurations, faults affecting `pmpaddr0` outnumber those on `pmpcfg0`, suggesting that the address registers offer a larger attack surface. Ibex is the exception, with comparable fault counts on both registers. Indeed, the NAPOT encoding of `pmpaddr0` requires four arithmetic instructions, whereas `pmpcfg0` is set by a single `csrw` instruction (see Figure 3 in Appendix A.1).

4.2 PMP Register Fault Effect at Bit Level

We now expand our analysis which has so far focused on the effects of fault in PMP registers in order to study bit-flips based on their location within the PMP register. Tables 4 and 5 report the spatial distribution of single, double, and multiple bit-flips across the bits of PMP registers for all evaluated hardware platforms in both groups G2 and G3. The first three columns define the hardware environment and the fault category, including the total number of successful fault injection samples n . Each value represents the percentage of occurrences relative to the total number of fault injection points n that produced the corresponding fault effect. Empty cells indicate either zero occurrences or categories with no available samples.

Results highlight an asymmetric vulnerability pattern linked to the underlying microarchitecture. The microarchitecture of the three RISC-V cores (CVA6, VexRISCV, Ibex) have a strong influence on the bit-flip location distribution.

Table 4 shows the different bit-flip locations across all evaluated RISC-V cores on `pmpaddr0`. The *Initial value* row establishes the golden execution baseline

Table 4: Percentage of `pmpaddr0` bit modifications for G2 & G3, across platform, cores, and fault categories. Fault categories are *Single*, *Double*, and *Multi* bit(s)-flip; n gives the total number of faulty executions. Initial configuration corresponds to `base_addr = 0x20000600` (128-byte region). **Bit fields:** CVA6 implements a 64-bit `pmpaddr0` register within bits 53–5 encode `base_addr` (physical address bits [55:2]); bits 63–54 are `WPRI` (reserved, always zero). Bits 4–0 encode `region_size` in NAPOT format (31–0 rows only). VexRISCV and Ibex implement 32-bit registers and occupy a single row (31–0).

Core	Fault cat.	Range	Bit position																																																				
			base_addr																												region_size																								
			31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																					
Initial value	63-32	WPRI											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																
	31-0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1																				
CVA6	Single (n = 408)	63-32	WPRI																																																				
		31-0			2		3																										19	69																					
	Double (n = 48)	63-32	WPRI																																																				
		31-0	4			2	42						6						1						27	4	38	2	15		12			2	6																				
	Multi (n = 959)	63-32	WPRI											15	23	18	18	16	21	20	15	20	17	20	16	20	21	16	16	18	18	20	25	20	22	18	39	44	46	43	50														
		31-0	21	16	34	23	37	16	21	15	25	20	16	24	24	25	21	16	26	21	20	22	30	27	31	36	45	19	20																										
VexRISCV	Single (n = 2)	31-0																																																		100			
	Double (n = 2)	31-0																																																		100			
	Multi (n = 33)	31-0	15	15	15	52	21	58	18	21	18	15	15	15	15	12	15	12	12	18	21	15	18	24	55	21	64	21	21	21	67	55	61	61																					
Ibex	Single (n = 59)	31-0												11	29																														19				2						
	Double (n = 118)	31-0	9												90					9																										90				1	1				
	Multi (n = 515)	31-0	16	14	15	18	21	38	35	49	18	45	39	41	17	33	33	32	15	32	35	34	14	14	25	15	61	22	30	13	61	61	55	59																					

before fault injection, where the register contains at bits [31:5] for RV32 and at bits [53:5] for RV64 encode the base address `0x20000600`, while bits [4:0] encode the region size under the NAPOT addressing mode.

We observe similar sensitive bit areas, for instance between [29:26], and between [11:0], which primarily represent bit-flips from 1 to 0 on CVA6 RISC-V core. Relative to their number, bits [4:0] are over-represented among isolated bit-flips on all three cores, although the effect is far weaker on Ibex, where they account for 21% of the flips for 5 of the 34 bits.

In particular, flipping bit 0 to zero reduces the encoded size field to its minimum value. This constrains the protected memory region to only 8 bytes, drastically shrinking the area covered by the PMP entry and potentially leaving large portions of the intended protected region exposed. We observe that this specific case is highly reproducible on the CVA6 RISC-V core, accounting for over 69% of single bit-flips. Furthermore, we observe a limited impact of single bit-flip faults on the `base_addr` field across the RISC-V cores. In the cases under study, a bit-flip in the least significant bits (LSB) would shift the base address by a small offset, whereas a bit-flip in the most significant bits (MSB) would result in a large displacement of the memory region protected by the PMP. Notably, the Ibex platform exhibits the highest susceptibility to bit-flip faults, with 29% and 32% of successful injections targeting the least significant and most significant bits of the address field, respectively.

Double bit-flip faults are observed more frequently than single bit-flips. In the majority of cases, double bit-flip injections impact the `base_addr` field across the different hardware platforms, affecting both the least significant and most significant bits of the address. On the CVA6 platform, a 1% occurrence of double bit-flips on bit 49 is sufficient to produce a complete displacement of the protected memory region, potentially allowing an attacker to bypass or escape the PMP-enforced boundary entirely. In contrast, the VexRISCV and Ibex processors exhibit a higher probability of double bit-flips affecting the most significant bits of the address beyond bit 16, with success rates of 100% and 90%, respectively. This suggests that these platforms are particularly vulnerable to large-offset address corruptions through double bit-flip fault injection.

Finally, multiple bit-flips (> 2) remain the most common occurrences. These are particularly dangerous as they allow both the base address and the size of the protected area to be simultaneously altered across all evaluated RISC-V cores. We highlight that the bits within the `region_size` field are the most frequently affected by multi-bit flip faults, a finding consistently observed across the RISC-V cores.

These bit-flips on `pmpaddr0` have a direct impact on the various software-based memory isolation. For instance, in Class 1, a single bit-flip shrinking a `NAPOT` region to 8 bytes forces the vast majority of the enclave’s memory outside the physical protection boundary. This allows the untrusted host OS to gain unauthorized read/write access to private code and data, completely breaking confidentiality and integrity. Similarly, in Classes 2 and 3, multiple bit-flips shifting the base address in `pmpaddr0` alter the physical boundaries of isolated zones. This shifting causes memory regions to overlap, enabling an attacker in one zone to arbitrarily read or corrupt the memory of an adjacent zone. The vulnerability also extends to RTOSes (Class 4); since it relies on the `TOR` addressing mode with TockOS, modifying a `pmpaddr0` register shifts the exact boundary separating two contiguous processes, allowing an unprivileged process to break isolation.

Table 5 presents the percentage of bit-field modifications within the `pmpcfg0` register across the evaluated RISC-V cores, and fault categories. Each cell reports the percentage of faulty executions in which the corresponding bit-field deviated from its initial value, where blank cells indicate zero occurrences. The initial configuration corresponds to a locked ($L=1$), read-only ($R=1$), `NAPOT`-addressed PMP entry ($A=11$), with execute and write privileges disabled ($X=0$, $W=0$).

Considering single bit-flip faults across RISC-V cores, the results reveal a heterogeneous susceptibility profile. On the CVA6 core, the `W` bit remains the most affected field (50%), followed by the `R` bit with a downward transition in 33% of cases. In contrast, the Ibex core on the same platform exhibits a markedly different profile, with single bit-flips concentrating on the `X` bit (74%) and the `A[0]` field (22%), leaving the write and read privileges unaffected. Notably, the VexRISCV core shows complete resilience to single bit-flip faults, recording zero successful injections under this category.

Table 5: Percentage of bit-field modifications in the `pmpcfg0` register for groups G2 & G3, across platform, cores, and fault categories. Fault categories are *Single*, *Double*, and *Multi* bit(s)-flip;; n gives the total number of faulty executions. Initial configuration corresponds to $L=1$, $A=NAPOT$, $R=1$ (read-only, locked, NAPOT). **Bit fields:** lock bit L (bit 7), two reserved bits $Bit6$ – $Bit5$, two-bit address-matching mode $A[1:0]$, privilege flags X (execute), W (write), and R (read).

Core	Fault category	Bit field							
		L	Bit6	Bit5	A[1]	A[0]	X	W	R
Initial value		1	0	0	1	1	0	0	1
CVA6	Single ($n = 18$)	—	11	—	—	—	6	50	33
	Double ($n = 2$)	—	—	—	—	—	100	100	—
	Multi ($n = 72$)	89	8	11	96	18	82	76	19
VexRISCV	Single ($n = 0$)	—	—	—	—	—	—	—	—
	Double ($n = 0$)	—	—	—	—	—	—	—	—
	Multi ($n = 4$)	50	25	25	50	—	75	75	—
Ibex	Single ($n = 369$)	4	—	—	—	22	74	—	—
	Double ($n = 175$)	6	—	—	15	13	78	67	3
	Multi ($n = 530$)	72	—	—	65	75	29	6	73

Under double bit-flip fault conditions, the most critical observation is recorded on the CVA6 RISC-V core, where 100% of the $n = 2$ successful injections simultaneously flip both the X and W bits upward. Since the initial state already grants read privilege ($R=1$), this fault unconditionally elevates the memory region to full XWR privileges, representing a complete breakdown of the access control policy. On the Ibex core, double bit-flips primarily affect the X bit (78%) and W bit (67%), representing a complete breakdown of the access control policy like on the CVA6. While the VexRISCV core records no successful injections in this category.

Multi bit-flip faults (> 2) represent the most destructive fault category and consistently produce the highest modification rates across RISC-V cores. The lock bit L emerges as the most frequently affected field, exhibiting downward transitions in up to 89% of cases on the CVA6, and between 50% and 75% across the remaining both RISC-V cores. Deactivating this bit is particularly critical as it permits lower-privilege software to overwrite the M-mode enforced protections at runtime. Concurrently, the addressing mode field $A[1:0]$ is heavily targeted, with downward transition rates reaching 96% on the CVA6, potentially switching the PMP entry to the `OFF` state or forcing a reinterpretation under `TOR` mode. The privilege flags X and W also show high upward transition rates, particularly on the CVA6 and Ibex cores, further amplifying the security impact of multi bit-flip faults. In contrast, the VexRISCV core exhibits limited exposure in this category ($n = 4$), though the observed faults still affect the L bit and the X/W privilege flags.

These hardware-level bit-flips on `pmpcfg0` lead to catastrophic failures when propagated to real-world from isolated software execution environments. For

Class 1 of Table 2, which rewrites PMP configurations during enclave execution boundaries, a well-timed fault can grant an active enclave illegitimate **XWR** privileges or clear the **L** bit, breaking isolation barriers between parallel enclaves. Similarly, in Class 2, a fault altering the privilege or addressing bits during a zone switch can erroneously expose neighboring peripheral registers or cross-zone memory segments. For Class 4, which configures and locks the PMP table exactly once at boot, injecting a single fault to drop the **L** bit or flip the **A** bits to 00 (i.e. **OFF**) completely strips away the firmware’s defense-in-depth layout, exposing the secure M-mode memory to unauthorized S-mode and U-mode operations. Finally, Class 4 presents the widest attack surface due to continuous context-switching; here, corrupting `pmpcfg0` privileges allows an unprivileged user process to inherit the access tokens of an adjacent secure process, silently breaking kernel-enforced application isolation.

In addition, simultaneous or sequential modifications to both `pmpcfg0` and `pmpaddr0` registers yield compound primitives that are significantly more powerful than their individual components. Elevating all **XWR** bits in `pmpcfg0` while expanding `region_size` in `pmpaddr0` grants full read/write/execute access over a larger-than-intended protected region. Likewise, switching the addressing mode to **TOR** in `pmpcfg0` while elevating **X/W/R** causes the active entry to cover all memory from 0x0 up to `pmpaddr0-1`, granting full privileges over the entire low-memory range. Class 1, this collapses the enclave boundary into the host OS address space; Class 2, it causes a zone to absorb neighboring memory zones; Class 3, it exposes the firmware image itself to S-mode with execute privileges; and Class 4, it allows a process to escape its region entirely and inherit the permissions of adjacent processes.

We also examine whether the initial values of PMP registers affect the resulting bit-flip patterns. The percentage tables in Appendix A.2 compare fault effects on `pmpaddr0` and `pmpcfg0` across three initial configurations. Results show that the initial value partially shapes fault localization. However, regardless of the initial value, fault injections leads to similar security-critical effects on the PMP configuration flow. The attack surface exposed by PMP fault injection is therefore independent to PMP registers initial values.

4.3 Discussion

Attack scenarios. Table 6 summarizes the exploitation scenarios identified across the four evaluated classes software-based of memory isolation, mapping each security-critical fault effect observed on the `pmpcfg` and `pmpaddr` registers to its corresponding impact on the memory’s isolation guarantees. Across all platforms, fault injection consistently produces at least one exploitable condition per Class, ranging from unauthorized privilege escalation to complete memory isolation breakdown. Notably, Class 1, 2 and 4 present the broadest attack surface due to its continuous context-switching model, while Class 3 is particularly vulnerable at boot time given its single-pass PMP configuration strategy.

Table 6: Exploitation scenarios exploiting modifications to `pmpcfg` and `pmpaddr` for software classes as described section 2.3.

Reg.	Fault Effect	isolated software execution environments			
		Class 1	Class 2	Class 3	Class 4
pmpcfg	L cleared (1→0)	Enclave boundary overwritable	Zone policy overwritable	Boot PMP overwritable	Process boundary overwritable
	L set (0→1)	SM cannot reconfigure enclave	Zone policy permanently frozen	Firmware locked at wrong config	Kernel loses region update ability
	A switched to OFF	Enclave unprotected	Zone entry disabled	M-mode memory exposed	Process region disabled
	A switched to TOR	Low memory exposed to enclave	Low memory covered by zone	Full low-memory layout exposed	Process gains access to low memory
	A switched to NAPOT	Enclave covers misaligned region	Zone boundary misaligned	Firmware covers wrong NAPOT block	Process region over unintended block
	X/W/R elevated (0→1)	Illegitimate XWR on enclave	XWR on neighboring segments	Firmware region executable/writable/readable	Process inherits adjacent privileges
pmpaddr	base_addr shifted	Host OS accesses enclave memory	Cross-zone memory overlap	Zone boundaries altered	Process escapes its region
	region_size reduced	Enclave memory outside boundary	Zone memory left unprotected	Firmware segments exposed	Process region shrunk
	region_size expanded	Enclave overlaps adjacent memory	Zone covers unintended segments	Firmware region extends into S-mode	Process gains access to neighbor
	addr set to 0x0	Enclave protection nullified	Zone entry covers no memory	Firmware protection disabled	Process has no fully protected region
	addr set to max	Enclave spans entire address space	Zone absorbs all physical memory	M-mode covers all physical memory	Process region engulfs all memory

Related work. Nashimoto et al. [18] first showed that PMP configuration registers can be altered by clock glitching on a single processor, and Queneherve et al. [21] analyzed fault propagation inside one core (CVA6). In contrast, our characterization on the *three RISC-V cores* and at *bit-level* shows that these effects are neither isolated nor core-specific. We trace how this low-level corruption propagates upward and breaks the isolation guarantees of all four PMP-based software stacks, from dynamic TEEs and zone-partitioning kernels to static firmware monitors and RTOS process isolation. To the best of our knowledge, this is the first analysis to connect bit-level PMP fault effects to the security of the full range of PMP-based software isolation layers.

Possible Countermeasures. Mitigating these fault effects requires defenses at both levels, since neither is sufficient alone.

At the hardware level, redundancy on `pmpcfg` and `pmpaddr` (e.g., DMR/TMR) or parity and error-correcting codes detect the single- and multi-bit corruptions we observed; the PMP Snapshot Engine [14] illustrates this direction, restoring parity-protected snapshots in one cycle and suppressing register-corruption and instruction-skip faults for 11.7% area overhead. Glitch detectors and clock/voltage monitors [23,12] further raise the difficulty of injection.

At the software level, the security monitor can verify the active configuration against a trusted reference at every context switch and trap on mismatch; jump address masking [19] targets this surface directly, XOR-masking jump addresses with a hash of the PMP registers so that a corrupted write diverts execution instead of silently leaving the enclave accessible. Temporal redundancy raises the attack cost without removing it, as instruction duplication and random delays only reduce the success rate of PMP-skipping faults [19], and conservative timing margins limit the attacker’s ability to target the exact update cycle. These mechanisms are complementary: robust protection combines hardware redundancy with software-level verification, as in [14].

5 Conclusion

This paper presented an experimental fault injection study targeting the RISC-V Physical Memory Protection mechanism across multiple cores. Our results demonstrate that clock glitching attacks can reliably corrupt both `pmpaddr0` and `pmpcfg0` registers, leading to critical memory isolation violations. We showed that these hardware-level faults translate into concrete exploitation scenarios on real-world isolated software execution environments based on PMP, including Dynamic TEEs, Zone partitioning, Static firmware monitors, RTOS, enabling privilege escalation, enclave isolation bypass, and unauthorized memory access. These findings challenge the assumption that PMP provides a robust hardware root of trust in adversarial environments, and highlight the need for fault-resilient designs and dedicated countermeasures in RISC-V-based trusted execution environments.

Future work will evaluate the overhead and fault coverage of these countermeasures, confirm the observed effects through localized injection vectors such as EMFI [17,16], and investigate their combination within a fault-tolerant PMP design for reliable RISC-V execution environments.

Acknowledgement. This work was co-funded by CREACH LABS supported by the french Directorate general of Armament (DGA) and the Brittany Region, France.

A Appendices

A.1 Address Calculation in NAPOT-Mode

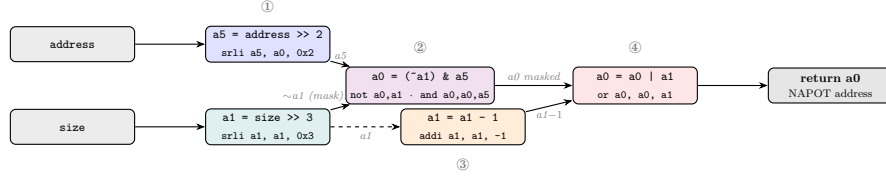


Fig. 3: Step-by-step data flow of the `addressingNAPOT` function, which encodes a base address and a region size into the NAPOT (Naturally Aligned Power-Of-Two) format expected by RISC-V PMP registers. ① The base address is right-shifted by 2 (`srli a5, a0, 0x2`) to drop the two lowest bits, while the size is right-shifted by 3 (`srli a1, a1, 0x3`) to extract the alignment mask. ② The complement of the size mask (NOT) is ANDed with the shifted address (AND), zeroing out the lower bits that encode the region size. ③ One is subtracted from the size mask (`addi a1, a1, -1`), producing a contiguous run of 1s in the low-order bits whose width encodes $\log_2(\text{size})$. ④ The masked address and the size suffix are combined with a bitwise OR (`or a0, a0, a1`), yielding the final NAPOT value where the upper bits hold the aligned base address and the lower bits encode the region size.

A.2 Percentage Bit-flip fault effect on PMP registers accross various Initial values

Table 7: Percentage of bit modifications in the `pmpaddr0` register relative to its programmed NAPOT value, across three initial configurations and three fault multiplicity categories on the Arty A7-100T platform (CVA6 core). CVA6 implements a 64-bit `pmpaddr` register; two rows are reported per fault category: `63-32` for the upper address bits and `31-0` for the lower bits. Fault categories are *Single*, *Double*, and *Multi* bit(s)-flip; n gives the total number of faulty executions. Initial configurations: ① `base_addr` = 0x20000600 (128-byte region), ② `base_addr` = 0x200006B0 (8-byte region), ③ `base_addr` = 0x20000700 (128-byte region). **Bit fields:** bits 53-5 encode `base_addr` (physical address bits [55:2]); bits 63-54 are `WPRI` (reserved, always zero). Bits 4-0 encode `region_size` in NAPOT format (`31-0` rows only).

Core	Fault cat.	Range	Bit position																																				
			base_addr																										region_size										
			31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
CVA6	Init. val.①	63-32	WPRI												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		31-0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	1	1					
	Single (n = 408)	63-32	WPRI												—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—			
		31-0	—	—	2	—	3	—	—	—	—	—	—	—	—	—	—	—	—	1	—	—	—	—	—	1	2	—	3	—	—	—	19	69					
	Double (n = 48)	63-32	WPRI												—	—	—	1	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	4			
		31-0	4	—	—	2	42	—	—	—	—	—	6	—	—	—	—	6	29	—	—	27	—	4	38	2	—	15	12	—	—	2	6						
	Multi (n = 959)	63-32	WPRI												15	23	18	18	16	21	20	15	20	17	20	16	20	21	16	16	18	20	25	20	22	18			
		31-0	21	16	34	23	37	16	21	15	25	20	16	24	24	25	21	16	26	21	20	22	30	27	31	36	45	19	20	39	44	46	43	50					
	Init. val.②	63-32	WPRI												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		31-0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	1	1	0					
	Single (n = 391)	63-32	WPRI												—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	1	3	—	—		
		31-0	—	—	1	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	29	8	17	3	—	26	1	11					
	Double (n = 113)	63-32	WPRI												—	—	4	—	15	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	1	1	9	
		31-0	4	—	—	2	3	—	1	4	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2	46	—	51	27	10	16	7	—					
	Multi (n = 2863)	63-32	WPRI												19	21	24	23	23	23	24	23	24	22	25	22	23	21	22	20	21	19	22	21	25	22			
		31-0	24	19	23	26	50	20	24	22	21	21	22	21	19	21	20	21	21	18	21	20	23	24	23	47	47	37	43	34	20	24	19	—					
	Init. val.③	63-32	WPRI												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		31-0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	1	1	1	1					
Single (n = 806)	63-32	WPRI												—	1	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—			
	31-0	—	—	—	5	2	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	10	—	4	3	4	—	—	—	—	71						
Double (n = 250)	63-32	WPRI												—	2	2	3	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—			
	31-0	1	—	—	18	28	—	—	—	—	—	—	—	—	—	—	—	—	—	—	2	—	2	2	14	41	46	—	2	—	4	7	28						
Multi (n = 3638)	63-32	WPRI												28	26	28	28	28	29	27	27	28	26	27	25	27	27	28	28	27	25	27	27	28	28				
	31-0	25	26	31	28	32	29	28	29	29	26	29	26	27	27	27	28	27	27	27	33	35	44	41	37	46	35	49	39	33	30	23	30						

Table 8: Percentage of bit-field modifications in the `pmpcfg0` register relative to its programmed value, across three initial configurations and three fault multiplicity categories on Arty A7-100T platform (CVA6 core). Each cell reports the percentage of faulty executions in which the corresponding bit-field deviated from its initial value. Fault categories are *Single*, *Double*, and *Multi* bit(s)-flip; n gives the total number of faulty executions. Initial configurations: ① $L=1$, $A=NAPOT$, $R=1$ (read-only, locked, NAPOT), ② $L=1$, $A=OFF$, $X=W=R=1$ (all permissions, locked, disabled), ③ $L=1$, $A=NAPOT$, $X=W=R=1$ (full-access, locked, NAPOT). **Bit fields:** lock bit L (bit 7), two reserved bits **Bit6–Bit5**, the two-bit address-matching mode $A[1:0]$, and the permission flags X (execute), W (write), and R (read).

Core	Fault category	Bit field							
		L	Bit6	Bit5	A[1]	A[0]	X	W	R
CVA6	<i>Init. val.</i> ①	1	0	0	1	1	0	0	1
	Single ($n = 18$)	—	11	—	—	—	6	50	33
	Double ($n = 2$)	—	—	—	—	—	100	100	—
	Multi ($n = 72$)	89	8	11	96	18	82	76	19
	<i>Init. val.</i> ②	1	1	1	0	0	1	1	1
	Single ($n = 23$)	—	9	—	—	21	—	70	—
	Double ($n = 8$)	—	100	—	—	12	—	88	—
	Multi ($n = 361$)	95	96	94	2	90	7	10	2
	<i>Init. val.</i> ③	1	1	1	1	1	1	1	1
	Single ($n = 13$)	8	—	—	—	—	54	38	—
	Double ($n = 16$)	—	81	19	12	—	75	—	—
	Multi ($n = 378$)	53	54	54	53	9	14	14	6

References

1. Freedom Metal – A bare-metal library for RISC-V hardware, <https://github.com/sifive/freedom-metal>
2. Ibex risc-v core, <https://github.com/lowrisc/ibex>
3. Multizone security, <https://github.com/hex-five/multizone-sdk>
4. OpenSBI: RISC-V Supervisor Binary Interface, <https://github.com/riscv-software-src/opensbi>
5. Vexriscv, <https://github.com/spinalhdl/vexriscv>
6. Volume 2, Privileged Specification version 20211203, <https://riscv.org/technical/specifications/>
7. Zephyr kernel architecture documentation: Memory protection, <https://www.zephyrproject.org/>
8. Arty A7 (2019), <https://digilent.com/reference/programmable-logic/arty-a7/reference-manual?redirect=1>
9. Breier, Jakub and Hou, Xiaolu : How Practical Are Fault Injection Attacks, Really? IEEE Access (2022). <https://doi.org/10.1109/ACCESS.2022.3217212>
10. Karaklajić, Duško and Schmidt, Jörn-Marc and Verbauwhede, Ingrid : Hardware Designer’s Guide to Fault Attacks (2013). <https://doi.org/10.1109/TVLSI.2012.2231707>
11. Bar-El, H., Choukri, H., Naccache, D., Tunstall, M., Whelan, C.: The sorcerer’s apprentice guide to fault attacks. Proceedings of the IEEE (2006). <https://doi.org/10.1109/JPROC.2005.862424>
12. Breier, J., Bhasin, S., He, W.: An electromagnetic fault injection sensor using hogge phase-detector. In: International Symposium on Quality Electronic Design (ISQED) (2017). <https://doi.org/10.1109/ISQED.2017.7918333>
13. Feng, E., Lu, X., Du, D., Yang, B., Jiang, X., Xia, Y., Zang, B., Chen, H.: Scalable memory protection in the PENGGLAI enclave. In: 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI ’21) (2021), <https://www.usenix.org/conference/osdi21/presentation/feng>
14. Larmann, C., Aljuffri, A., Marotzke, A., Garza, A., Hamdioui, S., Taouil, M.: The PMP snapshot engine: Fast and fault-resilient PMP reconfiguration for RISC-V. In: 2026 Design, Automation Test in Europe Conference (DATE) (2026). <https://doi.org/10.23919/DATE69613.2026.11539331>
15. Lee, D., Kohlbrenner, D., Shinde, S., Asanovic, K., Song, D.: Keystone: An open framework for architecting trusted execution environments. In: Proceedings of the Fifteenth European Conference on Computer Systems. EuroSys ’20 (2020)
16. Maistri, P., Po, J.: A low-cost methodology for em fault emulation on fpga. In: Design, Automation & Test in Europe Conference (DATE) (2022). <https://doi.org/10.23919/DATE54114.2022.9774507>
17. Nabhan, R., Dutertre, J.M., Rigaud, J.B., Danger, J.L., Sauvage, L.: A tale of two models: Discussing the timing and sampling em fault injection models. In: Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC) (2023). <https://doi.org/10.1109/FDTC60478.2023.00010>
18. Nashimoto, S., Suzuki, D., Ueno, R., Homma, N.: Bypassing Isolated Execution on RISC-V using Side-Channel-Assisted Fault-Injection and Its Countermeasure. IACR (2021). <https://doi.org/10.46586/tches.v2022.i1.28-68>
19. Nashimoto, S., Ueno, R., Homma, N.: Comparative analysis and implementation of jump address masking for preventing tee bypassing fault attacks. In: Proceedings

- of the 19th International Conference on Availability, Reliability and Security. Association for Computing Machinery (2024). <https://doi.org/10.1145/3664476.3664477>, <https://doi.org/10.1145/3664476.3664477>
20. Oleg, P.: The use of hardware security mechanisms (secure enclave) for secure storage of cryptographic keys on mobile devices. Universal Library of Innovative Research and Studies (2026). <https://doi.org/10.70315/uloap.ulirs.2026.0301014>
 21. Quénehervé, K., Pensec, W., Tanguy, P., Dafali, R., Lapôtre, V.: Exploring fault injection attacks on CVA6 PMP configuration flow. In: Euromicro Conference on Digital System Design (DSD) (2024). <https://doi.org/10.1109/DSD64264.2024.00015>
 22. Schuermann, L., Campbell, B., Ghena, B., Levis, P., Levy, A., Pannuto, P.: Tock: From research to securing 10 million computers. In: Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (2025). <https://doi.org/10.1145/3731569.3764828>, <https://doi.org/10.1145/3731569.3764828>
 23. Song, S., Tell, S.G., Zimmer, B., Kudva, S.S., Nedovic, N., Gray, C.T.: An fl-based clock glitch detector for security circuits in a 5nm finfet process (2022). <https://doi.org/10.1109/VLSITechnologyandCircuits46769.2022.9830157>
 24. Timmers, N., Spruyt, A., Witteman, M.: Controlling PC on ARM Using Fault Injection (2016). <https://doi.org/10.1109/FDTC.2016.18>
 25. Troughine, T., Bukasa, S.K.K., Escouteloup, M., Lashermes, R., Bouffard, G.: Electromagnetic Fault Injection Against a Complex CPU, toward new Micro-architectural Fault Models. Journal of Cryptographic Engineering (2021). <https://doi.org/10.1007/s13389-021-00259-6>
 26. Vasselle, A., Thiebauld, H., Maouhoub, Q., Morisset, A., Ermeneux, S.: Laser-Induced Fault Injection on Smartphone Bypassing the Secure Boot. In: Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC) (2017). <https://doi.org/10.1109/FDTC.2017.18>
 27. Zaruba, F., Benini, L.: The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FD-SOI technology (2019). <https://doi.org/10.1109/TVLSI.2019.2926114>, <https://github.com/openhwgroup/cva6>